

Mi trabajo DevOps explicado fácil

Una guía lógica, desde cero, con GitHub y la bitácora real del proyecto

QUÉ VAS A ENTENDER

Qué problema resolviste, cómo trabaja el equipo en GitHub, qué ocurre cuando suben código, cómo llega a Azure, cuándo intervenís vos y qué hicimos hasta hoy.

Microsoft Azure - Terraform - GitHub Actions - PostgreSQL - Event Grid

Versión 4 - 14/09/2026

1. ¿Qué problema viniste a resolver?

Antes, cada developer podía ejecutar el frontend y el backend en su computadora. Eso alcanza para programar, pero no para que todo el equipo pruebe la misma versión ni para demostrar un despliegue real.

Antes: todo local

Cada computadora podía tener versiones, configuraciones y bases distintas. Si algo funcionaba en una máquina no garantizaba que funcionara en otra.

Ahora: un ambiente compartido

Azure ofrece un lugar común. Todos prueban la misma versión, con la misma configuración y una base PostgreSQL administrada.

Tu trabajo no fue simplemente 'subir una página'. Construiste el camino completo entre el código del equipo y un ambiente compartido.

La analogía del edificio

Los developers diseñan las oficinas y los sistemas que se usarán adentro. DevOps prepara el edificio, la electricidad, las puertas, las llaves, el control de acceso y el procedimiento para habilitar una nueva versión sin romper lo que ya funciona.

Por eso DevOps se ocupa de Azure, Terraform, pipelines, secretos, despliegues, logs y recuperación. No decide cómo funciona una orden de trabajo: eso es aplicación. Sí decide cómo esa aplicación se construye, se publica y se opera.

2. Todo el sistema en una sola historia

Si recordás estas seis frases, ya entendés la estructura general.

1

El equipo escribe código
Frontend crea pantallas; backend crea reglas, endpoints y acceso a datos.

2

GitHub revisa automáticamente
Las pruebas y el build detectan problemas antes de mezclar el cambio con el resto.

3

Docker empaqueta una versión
La aplicación y sus dependencias quedan dentro de una imagen reproducible.

4

ACR guarda esa versión
La imagen se identifica con el SHA del commit para saber exactamente qué se desplegó.

5

Terraform prepara Azure
Describe qué recursos deben existir y muestra un plan antes de modificar nada.

6

Vos autorizás el despliegue
Primero aprobás el plan; después aprobás el apply; finalmente verificás el resultado.

En una frase

El developer entrega código; la automatización lo valida y empaqueta; Terraform prepara el lugar; DevOps decide cuándo esa versión puede ejecutarse en Azure.

3. Qué te pidieron y qué hiciste realmente

1. Resource Group y servicios

Creaste una organización clara en Azure y los servicios necesarios para ejecutar frontend, backend, PostgreSQL, imágenes, logs y eventos. No es solo una carpeta: define el límite operativo del ambiente.

2. Infraestructura como código

Todo quedó descrito con Terraform. Si mañana hay que reproducir o revisar la plataforma, el conocimiento no depende de recordar clics del portal.

3. Mensajería

Se creó Event Grid para distribuir eventos. El backend ya recibe endpoint y clave como secreto. No se inventaron consumidores porque los contratos todavía no están confirmados.

4. Pipelines

GitHub valida, construye imágenes y ejecuta Terraform. El plan y el apply requieren dos autorizaciones separadas de DevOps.

Resultado comprobado

Frontend HTTP 200; backend y proxy con database: up; Event Grid aceptó una publicación técnica HTTP 200; Terraform terminó con No changes.

Esto significa que las cuatro tareas DevOps están hechas. No significa que toda la aplicación funcional ya esté desarrollada.

4. Los recursos de Azure y para qué sirven

Pensá Azure como una pequeña ciudad que armamos para que viva el sistema.

Resource Group: el barrio

Agrupar los recursos de development. Permite ver costos, permisos y componentes dentro de un límite claro.

Container Apps: los locales

Uno ejecuta el frontend y otro el backend. Se abren cuando reciben tráfico y pueden quedar en cero cuando nadie los usa.

PostgreSQL: el archivo municipal

Guarda información persistente. Aunque reinicies el backend, los datos continúan en la base.

ACR: el depósito de versiones

Guarda las imágenes Docker. Cada etiqueta SHA permite recuperar una versión exacta.

Event Grid: el distribuidor de avisos

Recibe un evento y lo entrega a los consumidores suscriptos. Hoy existe el distribuidor; faltan destinatarios confirmados.

Log Analytics: la caja negra

Conserva registros para entender por qué una aplicación falló o qué ocurrió durante un despliegue.

Terraform State: el inventario

Guarda la relación entre el código Terraform y los recursos reales. Es sensible porque contiene IDs y secretos; por eso vive en un Blob privado y nunca se sube a Git.

5. El viaje de un cambio del frontend

Supongamos que Front agrega un filtro para buscar órdenes de trabajo.



1

Front crea feature/filtro-ordenes

El trabajo queda separado de develop para no afectar al resto mientras está incompleto.

2

Abre un Pull Request

GitHub instala dependencias, ejecuta lint, pruebas y build. Si algo falla, el cambio no debe fusionarse.

3

Se fusiona en develop

El equipo declara que la tarea está lista para integrarse con las demás historias.

4

Se construye una imagen

El pipeline crea una versión Docker y la guarda en ACR con el SHA del commit.

5

Terraform apunta a esa imagen

El plan muestra qué Container App se actualizará y confirma que no haya eliminaciones inesperadas.

6

DevOps autoriza

Vos aprobás plan y apply. Azure crea una nueva revisión del frontend.

7

Se prueba en development

El equipo abre la URL pública y verifica la historia junto con backend y PostgreSQL reales.

6. El pipeline no es un botón mágico

La automatización sirve para detener problemas temprano y dejar evidencia. Cada falla tiene una respuesta distinta.

Falla una prueba

No se fusiona. El developer corrige el código. DevOps no debe ocultar ni saltar el control.

El plan quiere destruir

Se detiene el despliegue. Primero se identifica por qué Terraform propone la baja y si existe riesgo de datos.

El apply falla

Se revisan logs y estado. No se vuelve a ejecutar a ciegas ni se modifica Azure manualmente sin documentarlo.

La app despliega pero no responde

Se ejecutan health checks y logs. Si la versión es defectuosa, se vuelve a una imagen anterior por SHA/digest.

PostgreSQL está apagado

El backend puede fallar en /api/health. Primero se verifica si fue un apagado intencional para ahorrar crédito.

Todo responde

Se ejecuta un plan posterior. No changes demuestra que Azure y Terraform quedaron alineados.

La lógica

Un pipeline exitoso solo demuestra que los pasos automáticos terminaron. Un despliegue se considera completo cuando además la aplicación responde y Terraform no detecta diferencias.

7. Por qué existen distintas ramas

Las ramas separan niveles de confianza. No todo código merece llegar inmediatamente a una versión estable.

feature/*: estoy trabajando

Puede cambiar y romperse. Nace desde develop y vuelve mediante Pull Request.

develop: el sprint integrado

Reúne historias terminadas. Es la fuente de development, donde el equipo prueba en conjunto.

release/vX.Y.Z: candidato

Congela el alcance del sprint. Solo recibe correcciones necesarias para que el PO pueda validar en test.

main: versión aprobada

Conserva releases cerradas. No es la rama diaria de trabajo.

¿Por qué no trabajar directamente en main? Porque se mezclarían experimentos, tareas incompletas y versiones aprobadas. Después sería difícil saber qué se probó y qué debería recuperarse ante un error.

Rama y ambiente no son lo mismo

La rama es una línea de código en GitHub. El ambiente es un conjunto de recursos ejecutándose en Azure. develop puede desplegarse en development, pero develop no es un servidor.

8. Qué es GitHub y qué guarda realmente

Git es el sistema que registra cambios. GitHub es el lugar compartido donde el equipo publica esos cambios, los revisa y deja evidencia de las decisiones.

Repositorio

Es la carpeta compartida del proyecto con su historia. Tenemos uno para Front, uno para Back y otro para infraestructura y documentación DevOps.

Commit

Es una foto identificada de un conjunto de cambios. Su SHA permite saber exactamente qué versión se revisó, empaquetó o desplegó.

Branch

Es una línea de trabajo separada. Una feature permite programar sin modificar inmediatamente la rama integrada del equipo.

Pull Request

Es un pedido para incorporar una rama en otra. Muestra diferencias, comentarios, aprobaciones y resultados automáticos.

Check

Es una validación automática: instalar, compilar, ejecutar pruebas, construir una imagen o validar Terraform.

Merge

Es la incorporación aprobada. El PR cerrado sin merge no cambia la rama destino; el merge sí crea un nuevo estado compartido.

La idea importante

GitHub no reemplaza Azure. GitHub guarda código y ejecuta automatizaciones; Azure ejecuta la aplicación. Una corrida verde tampoco prueba por sí sola que el sistema desplegado responda.

9. Cómo trabaja Front en GitHub

El cambio de configuración relativa muestra el recorrido completo sin necesidad de imaginar un ejemplo.



1

Actualizar develop

Front descarga el último estado compartido antes de comenzar.

2

Crear feature/config-api-relativa

El cambio queda aislado mientras se reemplazan referencias fijas por rutas /api/...

3

Hacer commits y push

Los commits registran avances; push los publica en GitHub para que el resto pueda verlos.

4

Abrir el PR hacia develop

GitHub compara la feature con develop. No debe apuntar a main porque main representa una versión aprobada.

5

Revisar y validar

El equipo mira el cambio y los checks. Si hay observaciones, se agregan nuevos commits a la misma rama.

6

Hacer merge

Cuando está aprobado, develop incorpora el cambio. Recién entonces forma parte de la versión integrada del sprint.

Estado verificado el 08/09/2026

La rama develop de Front ya existe. El PR #2 feature/config-api-relativa hacia develop sigue abierto. El PR #3 de integración con Backend ya fue fusionado en develop. DevOps tiene lectura en ese repositorio, por lo que la aprobación y el merge corresponden a sus propietarios.

En Back, el PR #3 con API y migraciones PostgreSQL está abierto contra develop y su CI terminó correctamente. Verde significa que pasó los controles definidos; todavía requiere revisión y merge del equipo.

10. De la decisión al primer ambiente

Esta bitácora cuenta qué problema apareció, qué decisión tomamos y qué quedó comprobado.

11/08 - Elegimos la plataforma

Se autorizó la fase DevOps. Elegimos Azure, Terraform, GitHub Actions y autenticación OIDC sin client secret. La región Brazil South fue rechazada por la política estudiantil y se cambió a Chile Central.

11/08 - Preparamos la base

Se crearon el almacenamiento del estado Terraform, ACR, identidad administrada, credenciales federadas y environments development/test con revisión obligatoria de DevOps.

13/08 - Primer despliegue

El PR de infraestructura se fusionó. DevOps aprobó plan y apply. Azure creó el entorno de Container Apps y Log Analytics. El workflow finalizó correctamente.

Qué significó

Ya existía el terreno operativo para desplegar servicios, pero todavía no estaban Front, Back, PostgreSQL ni mensajería. Infraestructura lista no equivale a aplicación terminada.

Primera lección

Cuando Azure rechazó una región o un permiso, no se ocultó el error. Se identificó la causa, se corrigió en Terraform y quedó registrada la decisión.

11. De PostgreSQL al trabajo actual

La segunda etapa convirtió la plataforma vacía en un ambiente development utilizable por el equipo.

25/08 - CI de aplicaciones

Se auditaron Front y Back. Se agregaron validaciones y construcción Docker. Back falló inicialmente porque mvnw no era ejecutable en Linux; el workflow se corrigió y quedó verde.

25/08 - PostgreSQL

Terraform creó PostgreSQL 16, la base obras_publicas, backups y secretos. Se protegió contra destrucción accidental y se verificó database: up desde el backend.

25/08 - Front y Back en Azure

Se publicaron imágenes por SHA en ACR y se desplegaron dos Container Apps. El proxy /api dio 502 por TLS SNI; se corrigió Nginx y luego funcionó directo y por proxy.

01/09 - Event Grid

Terraform creó el tópico CloudEvents 1.0. El backend recibió endpoint y clave como secreto. Un evento técnico fue aceptado con HTTP 200; no se inventaron suscripciones sin contratos.

08-14/09 - Coordinación

Front ya publicó develop: su PR #2 de rutas relativas sigue abierto y el PR #3 de integración fue fusionado. Back mantiene abierto el PR #3 de API y migraciones, con CI exitoso. El estado se volvió a verificar el 14/09; ambos PR abiertos todavía requieren el circuito de revisión correspondiente.

Resultado acumulado

Development tiene infraestructura, Front, Back, PostgreSQL, proxy y Event Grid. Los smoke tests y un plan posterior sin cambios demostraron salud y convergencia. La primera release hacia test todavía no fue ejecutada.

12. Development y test explicados fácil

Imaginá un restaurante: development es la cocina donde el equipo combina y ajusta; test es la mesa donde se presenta un menú completo para aprobación.

development

Cambia durante el sprint. Sirve para integrar frontend, backend, datos y eventos. Puede tener errores mientras se corrigen historias.

test

Recibe una release candidata estable. El PO valida el sprint completo y DevOps comprueba versión, logs y rollback.

1

Cerrar el alcance
El equipo confirma qué historias realmente terminaron.

2

Crear release/vX.Y.Z
Se parte desde el commit probado de develop.

3

Promover la imagen
Si el código no cambió, se usa el mismo digest que ya se probó. No se reconstruye sin motivo.

4

Autorizar test
DevOps revisa y habilita el despliegue del candidato.

5

Validar
El PO prueba funcionalmente; DevOps observa operación y recuperación.

6

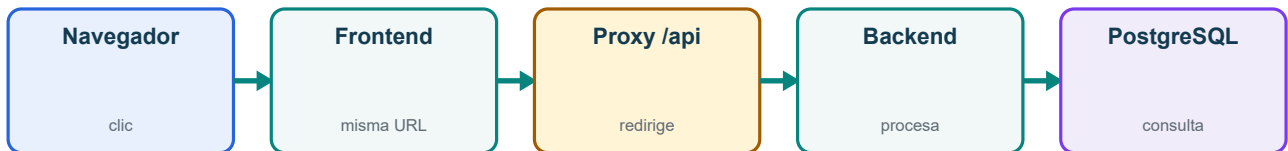
Cerrar
La release se fusiona en main, se crea el tag y las correcciones vuelven a develop.

Estado actual

Development está desplegado. El workflow de test está preparado, pero todavía no se ejecutó la primera release. Producción no está en el alcance actual.

13. Cómo viaja una llamada /api

Front hizo una observación correcta: si el navegador pide /api/ordenes desde el dominio del frontend, esa solicitud llega primero a Nginx, no directamente al backend.



En Azure ya configuraste ese proxy. Nginx toma cualquier ruta /api/... y la envía por HTTPS al dominio interno del backend. También se configuró TLS SNI para que Azure reconozca correctamente el destino.

Por qué Front usa rutas relativas

El código escribe /api/ordenes y funciona en distintos ambientes. Si escribiera una URL fija de Azure, quedaría atado a development y sería difícil promoverlo a test.

En local se conserva la misma regla: el código sigue llamando a /api. Vite puede tener un proxy de desarrollo que redirija hacia el backend local. Cambia la configuración del entorno, no el código de la aplicación.

Qué debe entregar Backend

Endpoints bajo /api, contratos JSON, códigos HTTP, errores, paginación, enums, validaciones y, cuando exista, autenticación por roles.

14. Backend, PostgreSQL y secretos

El navegador nunca debe conocer una contraseña de base de datos. Solo el backend conversa con PostgreSQL.

DB_URL

Indica dónde está PostgreSQL y qué base usar. Incluye TLS para cifrar la comunicación.

DB_USERNAME

Identifica al usuario técnico con el que se conecta el backend.

DB_PASSWORD

Se guarda como secreto de Container Apps. No aparece en Git, documentación ni frontend.

Migraciones

Cambian la estructura de la base de forma ordenada y versionada. Evitan editar tablas manualmente.

Ejemplo: si Backend agrega el campo prioridad a una orden, no alcanza con modificar una clase Java. También debe crear una migración que agregue la columna y probarla. De lo contrario, el código espera un dato que la base todavía no tiene.

Health check

/api/health ejecuta una consulta `SELECT 1`. Si responde database: up, demuestra que el backend está vivo y que puede conectarse realmente a PostgreSQL.

La regla de red actual permite conexiones desde servicios Azure para que Container Apps pueda llegar a PostgreSQL. Es aceptable para development, pero antes de producción debe reemplazarse por una solución de red privada.

15. Para qué sirve Event Grid

Un evento es como un aviso: 'algo ya ocurrió'. El emisor no llama uno por uno a todos los interesados; publica el aviso y el distribuidor lo entrega.



Ejemplo futuro: Obras Públicas aprueba un proyecto y publica un evento. Expedientes o Core podrían recibirlo sin que el backend de Obras conozca cómo están contruidos esos sistemas.

Lo que ya hiciste

Creaste egt-obras-publicas-dev-events, configuraste CloudEvents 1.0 y entregaste endpoint y clave al backend mediante secretos. Un evento técnico de prueba fue aceptado con HTTP 200.

Lo que todavía no corresponde hacer

No crear suscripciones, filtros ni eventos de negocio definitivos hasta que Core y los módulos confirmen nombres, payloads, consumidores y endpoints.

Si los contratos futuros requieren transacciones, orden estricto o detección de duplicados, habrá que reevaluar Service Bus. La herramienta se elige según la necesidad, no porque un nombre suene más empresarial.

16. Qué hacés vos cuando hay un despliegue

Ser DevOps no significa ejecutar un botón sin mirar. Tu rol es controlar el riesgo y dejar evidencia.

1

Confirmás qué se quiere desplegar
Rama, commit, imágenes y ambiente deben ser los correctos.

2

Revisás el resultado de CI
No se promueve una versión con pruebas o builds fallidos.

3

Autorizás el plan
Terraform consulta Azure y prepara el cambio propuesto.

4

Leés altas, cambios y bajas
Una eliminación inesperada es motivo para detenerse.

5

Autorizás el apply
Se aplica exactamente el artefacto que revisaste, sobre el mismo commit.

6

Ejecutás smoke tests
Frontend HTTP 200, /health, backend /api/health directo y por proxy, database: up.

7

Verificás convergencia
Un plan posterior debe terminar en No changes.

8

Registrás evidencia
Guardás workflow, SHA, plan, resultado y decisión de rollback.

Por qué hay dos aprobaciones

Aprobar el plan permite calcular y revisar. Aprobar el apply permite ejecutar. Separarlas evita aceptar cambios de infraestructura sin haber visto su impacto.

17. Qué significa 'apagar Azure'

Azure no tiene un interruptor único. Cada servicio tiene su propio modelo de consumo. El profesor tiene razón: en desarrollo conviene apagar lo que cobra aun cuando nadie trabaja.

Container Apps

Ya tienen `min_replicas = 0`. Sin solicitudes pueden escalar a cero y dejar de consumir cómputo.

PostgreSQL

Es el principal recurso a detener al terminar. Al pararlo deja de cobrarse cómputo, aunque siguen almacenamiento y backups.

Event Grid

Se cobra por operaciones. Si nadie publica eventos, su consumo es mínimo.

ACR y almacenamiento

Guardan imágenes y estado. Tienen costos pequeños o de almacenamiento y no deben borrarse para 'apagar'.

Atención con PostgreSQL

Azure vuelve a encender automáticamente un Flexible Server detenido después de siete días. Hay que volver a detenerlo o automatizar el control.

Para detenerlo:

Comando

```
az postgres flexible-server stop --resource-group rg-obras-publicas-dev  
--name psql-obras-publicas-dev-dfc86d
```

Para volver a trabajar:

Comando

```
az postgres flexible-server start --resource-group rg-obras-publicas-dev  
--name psql-obras-publicas-dev-dfc86d
```

Mientras PostgreSQL está apagado, el frontend puede abrir, pero el backend no podrá consultar datos y `/api/health` puede fallar. Eso es esperable, no necesariamente un despliegue roto.

18. Así debería trabajar el equipo de principio a fin

Este ejemplo junta desarrollo, GitFlow, Azure y tu rol en un solo recorrido.

1

09:00 - El equipo empieza
DevOps enciende PostgreSQL si estaba detenido. Cada developer actualiza develop.

2

09:15 - Se crean features
Front y Back trabajan en ramas separadas; la aplicación usa /api y secretos solo del lado backend.

3

Durante el día - Pruebas locales
Se ejecutan lint, tests, build y migraciones antes de abrir PR.

4

Se abre un PR
GitHub repite las validaciones. Los compañeros revisan el cambio y sus contratos.

5

Se fusiona en develop
El pipeline construye una imagen por SHA. Esa versión queda identificada.

6

DevOps despliega
Vos aprobás plan y apply. Después comprobás frontend, backend, base y proxy.

7

El equipo prueba en Azure
La historia se valida en el ambiente compartido, no solo en localhost.

8

Fin del día
Si nadie seguirá trabajando, se detiene PostgreSQL. Container Apps escalan a cero solos.

9

Fin del sprint
Se crea release/vX.Y.Z, se promueve a test y el PO valida el conjunto.

19. Quién hace qué

Los problemas aparecen cuando nadie sabe dónde termina su responsabilidad. Esta división evita confusiones.

Frontend

Pantallas, experiencia de usuario, rutas relativas /api, validaciones visuales, tipado de contratos y pruebas del cliente.

Backend

Reglas del dominio, endpoints /api, contratos JSON, autenticación, migraciones y publicación de eventos.

DevOps

Azure, Terraform, pipelines, imágenes, secretos, logs, despliegues, costo, smoke tests, promoción y rollback.

Product Owner

Prioriza el producto y valida funcionalmente la release. No reemplaza la autorización operativa de DevOps.

Una regla útil

Si el cambio modifica cómo funciona una orden, es aplicación. Si modifica cómo se construye, configura, despliega, observa o recupera la aplicación, es DevOps. Algunos cambios necesitan coordinación de ambos.

Ejemplo: Backend decide que una orden tiene prioridad HIGH. DevOps no inventa ese enum. Pero si Backend necesita una nueva variable o un secreto, debe informarlo y DevOps define cómo entregarlo de forma segura.

20. Qué falta y por qué no contradice lo hecho

La plataforma inicial está lista. Los siguientes pendientes pertenecen a otra capa o a una etapa posterior.

Aplicación funcional

Faltan endpoints, reglas, modelo y migraciones reales de Obras Públicas. Hoy el backend demuestra salud y conexión.

Contratos de eventos

Faltan acuerdos confirmados para crear suscripciones, filtros y consumidores de Event Grid.

Primera release en test

El workflow está preparado, pero todavía no se cerró un sprint con release/vX.Y.Z.

Red privada

Los accesos públicos temporales sirven para development. Producción necesitaría una arquitectura más restrictiva.

Presupuesto y alertas

Conviene definir un límite de crédito y avisos para detectar consumo inesperado.

Producción

No fue autorizada ni diseñada. No debe aparecer por inferencia solo porque development funciona.

Conclusión lógica

Tus cuatro tarjetas DevOps están terminadas porque sus recursos, automatizaciones y pruebas existen. La aplicación completa sigue pendiente porque requiere trabajo funcional del equipo y acuerdos externos.

21. Checklist simple que podés seguir siempre

Antes del despliegue

- ¿El PR está aprobado y todas las validaciones están verdes?
- ¿Sé qué commit e imágenes estoy desplegando?
- ¿El plan corresponde a development o test?
- ¿Hay alguna baja o cambio inesperado?

Después del despliegue

- ¿El frontend responde HTTP 200?
- ¿/health y /api/health responden?
- ¿El backend informa database: up?
- ¿/api/health funciona también desde el dominio del frontend?
- ¿Terraform termina en No changes?

Al terminar de trabajar

- ¿Alguien seguirá usando development?
- Si no, ¿PostgreSQL quedó detenido?
- ¿Quedó registrada la versión y el resultado?

Tu regla de oro

No decir 'está desplegado' solo porque GitHub quedó verde. Hay que comprobar la aplicación, sus dependencias y la convergencia de Terraform.

22. Glosario mínimo y evidencia

CI

Controles automáticos que se ejecutan antes de fusionar o publicar una versión.

CD

Proceso de preparar y desplegar una versión validada.

SHA

Identificador único de un commit; se usa para versionar imágenes.

Health check

Prueba técnica para saber si un servicio puede atender tráfico.

Smoke test

Prueba corta posterior al deploy para detectar fallos principales.

Drift

Diferencia entre lo que declara Terraform y lo que existe en Azure.

URLs actuales

Frontend: [ca-obras-publicas-dev-frontend...azurecontainerapps.io](#)

Backend health: [ca-obras-publicas-dev-backend.../api/health](#)

Evidencia

Event Grid: [PR #10 - Deploy 33501386215](#)

Guía base y reescritura: [PR #11](#) - [PR #12](#)

Trabajo actual: [Front PR #2](#) - [Front PR #3](#) - [Back PR #3](#)

La explicación final

El equipo crea una versión; GitHub comprueba que pueda construirse; Docker la empaqueta; ACR la guarda; Terraform prepara Azure; vos autorizás; Azure la ejecuta; los smoke tests confirman que funciona; y al terminar se apaga lo que consume crédito.